

Turn-Based Combat System Specification

Feature Requirements & Planning Guide

Modern Code Course

October 29, 2025

Contents

Contents	2
1 Combat System Specification	3
1.1 What We're Building: Turn-Based Combat System	3
1.1.1 System Overview	3
1.1.2 Required Features	3
1.1.3 Advanced Features (Optional)	5
1.1.4 Technical Requirements	9
1.1.5 Development Approach	9
2 Next Steps	11

1 Combat System Specification

This document describes the features and requirements for building a turn-based combat system. Use it to plan your system architecture with ChatGPT, understand what you need to build, and continue working on your project at home.

1.1 What We're Building: Turn-Based Combat System

Today's Goal

Build a complete turn-based combat system using top-down design and AI-assisted method planning.

This document describes *what* we want to build. Use it to plan your system architecture with ChatGPT, then implement following the exercises in the main document.

1.1.1 System Overview

A turn-based combat system is a game where a player and an enemy take turns performing actions until one is defeated. Think of classic RPGs like Final Fantasy or Pokemon battles.

Core Concept:

- ▶ Two combatants: Player and Enemy
- ▶ Alternating turns: Player acts, then Enemy acts
- ▶ Actions: Attack, Heal, Defend
- ▶ Game ends when one combatant's HP reaches zero

1.1.2 Required Features

Your combat system must implement the following features:

Game State Management

Health Points (HP):

- ▶ Both Player and Enemy start with maximum HP (100)
- ▶ HP decreases when attacked
- ▶ HP increases when healed (capped at maximum)
- ▶ Game ends when HP reaches zero

Combat Stats:

- ▶ Armor: Reduces incoming damage
- ▶ Armor resets to zero at the end of each turn
- ▶ Base damage: Standard attack damage before armor reduction

Turn Management:

- ▶ Turn counter tracks current turn number
- ▶ Each turn: Player acts → Enemy acts → Cleanup → Next turn

Player Actions

Action 1: Attack

- ▶ Deal damage to enemy based on base damage minus enemy armor
- ▶ Minimum damage is 0 (armor cannot increase HP)
- ▶ Display attack message and updated enemy HP

Action 2: Heal

- ▶ Restore HP to player
- ▶ Healing amount is fixed (20 HP)
- ▶ HP cannot exceed maximum (100)
- ▶ Display healing message and updated player HP

Action 3: Defend

- ▶ Increase player armor by fixed amount (10 armor)
- ▶ Armor bonus lasts for one turn only
- ▶ Reduces damage taken on enemy's next attack
- ▶ Display defense message and updated armor

Enemy AI

The enemy must make decisions automatically:

Enemy Strategy:

- ▶ If enemy HP < 30: Heal (prioritize survival)
- ▶ Otherwise: 90% chance to Attack, 10% chance to Defend
- ▶ Enemy follows same action rules as player

Console Input/Output

Note: This is a console-based application using text input/output. Use `IO.println()` for output and `IO.readInt()` for input.

Initialization:

- ▶ Print game title and welcome message to console
- ▶ Print starting HP for both combatants
- ▶ Print turn 1 beginning message

During Combat:

- ▶ Print current turn number to console
- ▶ Print Player HP and Enemy HP to console
- ▶ Print Player Armor and Enemy Armor to console
- ▶ Print action menu for player (1=Attack, 2=Heal, 3=Defend)
- ▶ Read player input using `IO.readInt()`
- ▶ Print action results with formatted messages to console

Game Over:

- ▶ Print winner announcement to console
- ▶ Print final turn number
- ▶ Print final HP for both combatants

Input Validation

Player Input:

- ▶ Prompt for action choice (1, 2, or 3)
- ▶ Validate input - must be 1, 2, or 3
- ▶ If invalid, re-prompt until valid input received
- ▶ Handle invalid input gracefully (no crashes)

1.1.3 Advanced Features (Optional)

These features enhance the basic combat system. They may force you to rethink some methods and implementations. Implement them if you are having fun with this exercise, OR you think you need the exercise. These are just some ideas, try implementing your own.

Critical Hits

Random Critical Damage:

- ▶ Attacks have 20% chance to be critical hits
- ▶ Critical hits deal 1.5x normal damage
- ▶ Display special message when critical hit occurs
- ▶ Applies to both player and enemy attacks

Enhanced Combat Feedback

Combat Log:

- ▶ Formatted console messages for all actions
- ▶ Clear indication of who performed what action
- ▶ Print damage dealt/healing received/armor gained to console
- ▶ Consistent console output formatting throughout

Console Status Display:

- ▶ Use blank lines or separators (e.g., "—") to separate turns in console output
- ▶ Easy-to-read HP and armor values printed to console
- ▶ Print action feedback immediately after player choice

Improved Enemy AI

Advanced Strategy:

- ▶ Enemy considers player's HP when deciding to attack vs defend
- ▶ Enemy times heals more strategically
- ▶ Vary enemy behavior based on combat situation

Game Balance Constants

Configurable Values:

- ▶ Use static final constants for all game values:
 - MAX_HP = 100
 - BASE_DAMAGE = 25
 - HEAL_AMOUNT = 20
 - DEFENSE_BONUS = 10
 - LOW_HP_THRESHOLD = 30
 - CRITICAL_CHANCE = 0.2 (20%)
 - CRITICAL_MULTIPLIER = 1.5
- ▶ Makes it easy to balance and adjust game difficulty

Status Effects System

Poison and Regeneration:

- ▶ Heal or Attack may apply a random status effect: Poison or Regeneration
- ▶ Status effects last for a fixed number of turns (use a turn counter)
- ▶ **Poison:** Deals damage over time each turn (e.g., 5 HP per turn for 3 turns)
- ▶ **Regeneration:** Heals HP over time each turn (e.g., 3 HP per turn for 4 turns)
- ▶ Track status effect duration using individual variables (e.g., `playerPoisonTurns`, `enemyRegenTurns`)
- ▶ Only one status effect active at a time per character
- ▶ Print status effect messages each turn ("Player is poisoned! -5 HP")

Combo System

Action Combos:

- ▶ Track the last action performed by each character (use variables like `playerLastAction`)
- ▶ Bonus damage/effects when the same action is used consecutively
- ▶ Example: "Combo Attack!" - 2nd consecutive attack deals extra 5 damage
- ▶ Example: "Combo Heal!" - 2nd consecutive heal restores extra 10 HP
- ▶ Reset combo counter when a different action is used
- ▶ Print combo notifications when triggered

Special Moves with Cooldowns

Powerful Abilities:

- ▶ Add a 4th action option: "Special Move" (cooldown-based ability)
- ▶ Track cooldown using a turn counter (e.g., `playerSpecialCooldown`)
- ▶ Special moves have powerful effects but can only be used every N turns
- ▶ Examples:
 - "Power Strike" - Deals 2x damage, 3 turn cooldown
 - "Massive Heal" - Restores 50 HP, 4 turn cooldown

- "Shield Wall" - Increases armor by 20 for 2 turns, 5 turn cooldown
- ▶ Show cooldown status: "Special Move available in 2 turns"
- ▶ Block special move if cooldown > 0

Battle Statistics Tracking

Combat Analytics:

- ▶ Track total damage dealt by player and enemy (use variables)
- ▶ Count number of critical hits, combos, special moves used
- ▶ Track longest combo chain
- ▶ Calculate average damage per turn
- ▶ Display statistics at game end:
 - "Total damage dealt: 180 HP"
 - "Critical hits: 3"
 - "Longest combo: 4"
 - "Average damage per turn: 18.0"
- ▶ Use this data to make the game more strategic

Multiple Rounds / Best of Series

Tournament Mode:

- ▶ Play multiple rounds until someone wins a "best of 3" or "best of 5"
- ▶ Track round wins (e.g., `playerRoundWins`, `enemyRoundWins`)
- ▶ Reset HP and stats between rounds
- ▶ Display overall score: "Player 2 - Enemy 1"
- ▶ Announce series winner when threshold is reached
- ▶ Option to play single match or tournament mode at start

Difficulty Levels

Adjustable Challenge:

- ▶ Let player choose difficulty at game start (Easy, Normal, Hard)
- ▶ Adjust game constants based on difficulty:
 - **Easy:** Enemy has lower HP, player deals more damage
 - **Normal:** Standard values
 - **Hard:** Enemy has more HP, player takes more damage, smarter enemy AI
- ▶ Use if/else or switch to set constants based on selection
- ▶ Display selected difficulty in combat status

Berserker Mode

Low HP Advantage:

- ▶ When HP drops below 25%, character enters "Berserker Mode"
- ▶ Berserker Mode effects:
 - Attacks deal 1.5x damage
 - Defense/Healing reduced by 20%
 - Special status message: "Player enters Berserker Mode! Damage +50%!"

- ▶ Track berserker state with boolean (`playerBerserker`, `enemyBerserker`)
- ▶ Apply berserker bonuses automatically when HP threshold reached
- ▶ Lose berserker mode if healed above threshold

Counter-Attack System

Defensive Reactions:

- ▶ When defending, there's a chance to counter-attack on the next turn
- ▶ Counter-attack tracks if previous action was defend (use boolean `wasDefending`)
- ▶ If defending last turn, 30% chance to automatically counter-attack before normal turn
- ▶ Counter-attacks deal 50% of normal damage but happen instantly
- ▶ Print: "Player counter-attacks! Enemy takes 12 damage!"
- ▶ Makes defending more strategic and valuable

Turn Order Variations

Speed/Initiative System:

- ▶ Add a speed stat to each character (e.g., `playerSpeed = 50`, `enemySpeed = 45`)
- ▶ Faster character goes first each round
- ▶ Speed can be modified by actions (defending increases speed next turn)
- ▶ If speeds are equal, randomize who goes first
- ▶ Display speed values in status: "Player Speed: 50 | Enemy Speed: 45"
- ▶ Makes combat more dynamic and less predictable

Action Prediction Bonus

Mind Games:

- ▶ Before enemy's turn, let player predict what enemy will do (Attack/Heal/Defend)
- ▶ If prediction is correct, player gets a bonus (extra damage, guaranteed crit, etc.)
- ▶ Track prediction with a variable (`predictedEnemyAction`)
- ▶ Check prediction accuracy after enemy's turn
- ▶ Print: "Correct prediction! Your next attack deals +10 bonus damage!"
- ▶ Adds a strategic element of reading your opponent

1.1.4 Technical Requirements

Method-Based Design:

- ▶ Use methods to organize code following Single Responsibility Principle
- ▶ Main method should orchestrate the game loop
- ▶ Helper methods for calculations and actions
- ▶ Each method should have a clear, single purpose

Code Quality (Hints)

Documentation:

- ▶ Javadoc is optional but it may help YOU with your methods
- ▶ @param tags for parameters
- ▶ @return tags for return values
- ▶ Inline comments for complex logic

Design Principles:

- ▶ Single Responsibility Principle: Each method does one thing
- ▶ DRY (Don't Repeat Yourself): Reuse methods, avoid duplication
- ▶ KISS (Keep It Simple, Stupid): Simple, clear solutions
- ▶ Meaningful names: Clear method and variable names

1.1.5 Development Approach

Top-Down Design:

- ▶ Start with the main game loop structure
- ▶ Plan the big picture first (grand strategy)
- ▶ Then fill in method implementations
- ▶ Finally, refactor and improve

Progressive Development:

- ▶ Phase 1: Make it work (inline logic)
- ▶ Phase 2: Make it better (extract methods)
- ▶ Phase 3: Make it great (enhancements, testing)

Using This Specification

How to use this document:

1. Review the required features above and make a plan
2. You can use ChatGPT to plan your system architecture:
 - ▶ "I need to build a turn-based combat system. Help me plan the overall architecture - what are the main parts?"
 - ▶ "What is the high-level flow of a turn-based combat game?"
 - ▶ "How should I structure my game loop?"
3. Create a method map showing:
 - ▶ Main game loop structure
 - ▶ Key methods needed
 - ▶ Responsibilities of each method
4. Implement following top-down approach:
 - ▶ Start with game loop skeleton
 - ▶ Fill in method implementations
 - ▶ Refactor and improve

2 Next Steps

After reviewing this specification:

1. Use ChatGPT to plan your system architecture based on these requirements
2. Create a method map showing your planned structure
3. Follow the main exercise document for step-by-step implementation guidance
4. Use this document as reference when building at home

Remember

This document describes *what* to build.

The main exercise document shows you *how* to build it using:

- ▶ Top-down design approach
- ▶ AI-assisted planning
- ▶ Progressive development (work → better → great)
- ▶ Refactoring techniques